Department of Computer Science

Banasthali Vidyapith

Software Requirement Specifications (SRS) and Software Design Specifications (SDS)

VEDIC HUMAN - YOGA AT EASE

G- 11

Guided By:

Mr. Sushil Kumar Ray

Submitted By:

Aanvi Rawat 251H0660001
Natasha Pundir 251H0660044
Sakshi 251H0660057

Chapter 1 SRS

TABLE OF CONTENTS

1. Introduction

1.1 Purpose…… 04
1.2 Scope…… 04
1.3 Definitions, Acronyms and Abbreviations…….. 04
1.4 Overview……..... 05

2. General Description

2.1 Product Perspective…….....….. 05
2.1.1 Product Function …….....…. 06
2.1.2Hardware Interface……..... … 07
2.1.3Software Interface…….....……..08
2.2 Communication Interface…….....……...08
2.3 User Characteristics……...……08
General Constraints…….....……....08
2.4 Technologies used……...…….....09

3. Specific Requirements

3.1 Functional Requirements……..11
3.2 Non-Functional Requirements ……....….11
3.2.1 AVAILABILITY…….....……11
3.2.2 SECURITY…….....……....11
3.2.3 RELIABILITY…….…….....……..12
3.2.4 PORTABILITY…….....……..12
3.2.5 MAINTAINABILITY…….....12

1 Introduction

Vedic Human is an AI-based yoga training system that enables users to practice yoga with posture guidance. The system shall allow users to register and log in, start yoga sessions, detect poses in real time using a webcam, provide corrective feedback, and store session progress for performance tracking through a web-based interface.

1.1Purpose

This Software Requirements Specification (SRS) defines the objectives and features of VedicHuman, an AI-powered web application for real-time yoga pose detection and feedback. The SRS serves as a "blueprint" for developers, testers, and stakeholders, detailing how the software should perform and interact. It ensures clarity on functionality (pose detection, feedback, progress tracking), guiding implementation and testing.

1.2 Scope

VedicHuman is a browser-based yoga trainer that uses the user's webcam to detect body pose and provide corrective feedback. It will support multiple asanas (yoga poses) and compute an accuracy score for each pose. Key features include real-time visual feedback on alignment, and user progress tracking (best scores, session history, practice streaks). It doesn't handle live instructors or remote classes; rather it is a standalone self-practice tool. The system will use HTML/CSS/JavaScript for the UI, a Flask backend with SQLite for data storage, and TensorFlow's MoveNet model for pose estimation.

1.3 Definitions, Acronyms, and Abbreviations.

- **VedicHuman:**
  The name of the AI-based yoga training web application is developed to provide posture guidance and progress tracking.

- SRS/SDS: *Software Requirements Specification* and *Software Design Specification*. The SRS outlines *what* the system should do; the SDS details *how* it is designed.

- Asana (Yoga Pose): A specific body posture in yoga (e.g., Tree Pose, Warrior).

- MoveNet: A TensorFlow pose-detection model. MoveNet is an "ultra-fast and accurate" neural network that detects 17 body keypoints in real time (30+ frames/sec).

- Flask: A lightweight Python web framework for building web applications.

- SQLite: A serverless, self-contained SQL database engine. It is small, fast, and reliable (widely used in embedded/mobile apps).
- Frontend: The web-based user interface (HTML/CSS/JavaScript).
- Backend: The server-side Flask application and AI model.
- User Session / Streak: A session is one practice attempt (time-bound). A streak is a count of consecutive days with at least one session.

1.4 Overview

This document describes the overall requirements of the Vedic Human system. Section 2 (General Description) explains the system's context, user interface, and technology stack. Section 3 (Specific Requirements) lists functional and non-functional requirements. Chapter 2 (Design Specification) (the SDS) covers the system architecture, data design, and UI design. References to guiding sources and examples are included to ensure accuracy and completeness.

2. The Overall Description

This section provides a general overview of the Vedic Human system and the factors that influence its requirements. It describes the system's operating environment, users, constraints, and major features. This section helps customers understand the system at a high level, while detailed functional specifications are defined in later sections.

2.1 Product Perspective

Vedic Human is a new, standalone web application (no legacy system integration). It provides a browser-based interface for guided yoga practice with AI feedback. The product fits within the category of fitness/health applications, but with real-time computer-vision feedback. The system consists of:

- **User Equipment:** A standard PC or laptop with a webcam and modern browser (e.g. Chrome, Firefox). No special hardware is needed. The webcam captures live video of the user performing yoga poses.
- **Server/Cloud:** A Flask-based server (could be local or cloud-hosted) that runs the MoveNet model for pose estimation, stores data in SQLite, and serves the web pages.
- **Interfaces:** The browser frontend interacts with the webcam API and the Flask backend via HTTP. The backend calls TensorFlow (MoveNet) for pose analysis.

2.1.1 Product Function

Key functions of Vedic Human include:

- **UserAccounts (Authentication):** Users can create an account and log in so that personal progress can be tracked.

- **Pose Training Mode:** Users select a yoga pose or sequence to practice. The app provides step-by-step instructions or a tutorial for each selected pose (e.g. Tree Pose guidance).

- **Real-Time Pose Detection:** Once training starts, the system captures live video from the webcam and applies the MoveNet pose model. It identifies body landmarks (e.g. shoulders, elbows, hips) in each frame.

- **Pose Classification and Scoring:** The detected landmarks are converted to joint angles and compared to a reference pose. The system classifies which pose is being performed and computes an accuracy score (percentage alignment). For example, it measures deviations in angles and assigns a numeric "pose score".

- **Visual Feedback:** The app overlays the live video feed with feedback cues. Correctly aligned joints may light up green, while misaligned ones are marked red. Real-time visual guidance helps the user adjust posture. This color-coding (green vs. red) follows established pose-correction conventions.

- **Session Data Logging:** Each practice session's data is recorded: the pose performed, date/time, achieved score, duration, and any corrective feedback messages. This is stored in the SQLite database.

- **Progress Tracking:** The app provides a "Progress" dashboard that shows the user's best score for each pose, their session history (dates and scores), and their current streak of practice days. For example, it highlights the *highest* accuracy score achieved in a pose and counts how many consecutive days the user has exercised. (These features motivate continued practice.)

- **Content and Tutorials:** The system includes static reference content. For each pose, it can display an instructional page with steps or images. It also has a main menu/homepage where users can start a new training or access tutorials.

2.1.2 Hardware Interfaces

The Vedic Human system interacts with limited hardware components to perform its intended functionality. The primary hardware interface is a **webcam**, which is used to capture live video input of the user during yoga sessions. The software accesses the webcam through the web browser and operating system's standard media access mechanisms. No direct control over the hardware is required beyond video capture. The system also interacts with standard computing hardware such as a personal computer or laptop to process user input and display output through the screen and audio devices. Audio output devices such as speakers or headphones are used to deliver voice feedback to the user when enabled. The system does not require

interaction with specialized sensors, wearable devices, or external hardware controllers. All hardware communication is handled using existing system and browser-supported protocols, and no proprietary hardware interfaces are involved.

2.1.3 Software Interfaces

This section describes the software products and interfaces that are required for the proper functioning of the **Vedic Human** system. This software components interact with each other to provide real-time yoga pose detection and user management.

Frontend Interface:

Purpose:
The front-end interface is used to interact with the user. It allows users to register, log in, start yoga sessions, view feedback, and check progress.

Interface Description:
The frontend is developed using HTML, CSS, and JavaScript. It communicates with the backend server using HTTP requests. User actions such as button clicks, form submissions, and webcam access are handled through JavaScript.

Backend Interface

Name: Flask Web Server
Mnemonic: API
Source: Python community

Purpose:
The backend interface processes requests from the frontend, manages user sessions, and connects with the AI model and database.

Interface Description:
The backend is implemented using Python and Flask. It receives requests from the frontend in JSON format and sends responses back after processing. It also controls the execution of the AI pose detection model.

AI Model Interface

Name: TensorFlow MoveNet
Mnemonic: AI-POSE
VersionNumber: TensorFlow MoveNet (pre-trained model)

Purpose:

The AImodel interface is responsible for detecting human body key points and analysing yoga poses in real time.

InterfaceDescription:

The backend sends video frames captured from the webcam to the MoveNet model. The model processes these frames and returns key point coordinates, which are used to evaluate posture accuracy.

Database Interface

Name:	SQL Database
Mnemonic:	DB
Version Number:	SQLite

Purpose:

Thedatabaseinterface is used to store and retrieve user information, yoga session records, and performance data.

Interface Description:

Thebackendcommunicates with the database using SQL queries. Data such as login credentials, session duration, accuracy scores, and progress history are stored and retrieved as required

All software interfaces work together to ensure smooth interaction between the user interface, backend logic, AI processing, and data storage. This integration enables real-time yoga to pose detection and personalized feedback.

2.2 User Characteristics

The Vedic Human system is intended for students, beginners, and regular yoga practitioners. Users are expected to have basic knowledge of using a computer, web browser, and webcam. No technical or programming expertise is required. The system is designed with a simple and user-friendly interface so that users of all age groups can easily navigate, start yoga sessions, receive posture feedback, and view their progress without difficulty.

2.3 Constraints

- The system requires a stable internet connection for smooth client–server communication.
- Yoga pose detection accuracy depends on webcam quality, camera angle, and lighting conditions.
- The application requires a minimum hardware configuration to run AI processing efficiently.
- The system supports a limited number of yoga poses based on the trained AI model.
- Performance may vary on low-end systems or during parallel application usage.

2.4 Technologies Used

- **HTML** isusedtocreatethe structure of web pages such as login, home, and session pages.
- **CSS** isusedtodesignand style the user interface for better usability.
- **JavaScript** isusedtohandle user interactions, webcam access, and frontend logic.
- **Python** isusedasthemain backend programming language.
- **Flask** isusedtohandleserver-side processing and communication between frontend and backend.
- **TensorFlow** isusedtorun the AI-based pose detection model.
- **MoveNet** isusedforreal-time human pose estimation.
- **SQL (SQLite/MySQL)** is used to store user data, session history, and progress details.

3. Specific Requirements

3.1 Functional Requirements

The functional requirements define the specific actions the Vedic Human system must be able to perform.

3.1.1 User Management & Authentication

- **Registration**: The system shall allow users to create accounts by providing a username, email, and password.
- **Login**: The system shall authenticate registered users using their username and password before granting access.
- **Dashboard**: Upon successful login, the system shall display a personalized home page/dashboard.
- **Logout**: The system shall provide a secure logout option to terminate the user session.

3.1.2 Yoga Session Management

- **Session Initiation**: Users shall be able to start a new yoga session directly from the home page.
- **Pose Selection**: The system shall allow users to choose a specific yoga pose from a supported list before starting.
- **Webcam Activation**: The system shall automatically activate the user's webcam to capture real-time video during the session.

3.1.3 AI Pose Detection & Analysis

- Real-time Detection: The system shall identify human body keypoints (landmarks) from the live video feed using the MoveNet model.
- Posture Analysis: The system shall compare the user's detected posture against reference posture data in real time.
- Continuous Monitoring: The system shall evaluate posture accuracy continuously throughout the duration of the active session.

3.1.4 Feedback & Guidance

- **Visual Feedback**: The system shall provide real-time visual indicators (such as color-coded joint overlays) to help users correct their posture.
- **Instructional Text**: The system shall display text-based guidance and session information, such as duration.
- **Voice Guidance**: The system may provide optional voice-based feedback to assist users during practice.

3.1.5 Progress Tracking & Library

- **Data Logging**: The system shall record and store session details, including the date, duration, and achieved accuracy score.
- **User History**: The system shall allow users to view their past sessions and overall progress metrics.
- **Pose Information**: The system shall provide descriptions and tips for each supported pose to help users improve.

3.2 Non-Functional Requirements

The non-functional requirements define the quality attributes and constraints of the system.

3.2.1 Availability

- The application should be available 24/7 for user practice, excluding scheduled maintenance windows.
- The system should start up and be ready to process requests within a few seconds of loading.

3.2.2 Performance

- **Processing Speed**: The system must process video frames in real time, with 90% of pose estimation results returned within 100ms per frame.

- **UI Responsiveness**: Standard user interface actions, such as page loads, should respond within 2 seconds.

3.2.3 Security

- **Data Protection**: User passwords must be stored securely using encryption in the SQLite database.
- **Access Control**: Only authenticated users shall have access to personal data and session features.
- **Encryption**: If deployed over the internet, the system should utilize HTTPS for secure communication.

3.2.4 Reliability

- The system should be capable of handling at least 100 concurrent sessions without data loss.
- Database writes for session results must be atomic to prevent data corruption during a system crash.

3.2.5 Portability

- The backend software shall be compatible with Windows, Linux, and macOS environments.
- The client interface must support all major modern web browsers, including Chrome, Firefox, and Edge.

3.2.6 Usability & Maintainability

- **Interface Design**: The UI must be intuitive with clearly labeled buttons and color cues that follow established conventions (e.g., green for correct, red for incorrect).
- **Modularity**: The codebase must be modular, separating camera handling, AI processing, database management, and UI logic for easier maintenance.

Chapter 2 SDS

TABLE OF CONTENTS

1. **Introduction**

1.1 Purpose ……14
1.2 Scope……14
1.3 Terms, Definitions, Acronyms and Abbreviations……..14
1.4 Overview of Document ……..……...…..15

2. **System Architectural Design**

2.1 High Level Design Overview……...….. 16

2.2 Detailed Description of Components..... 17
2.2.1 Structure Chart …….....….... 18
2.2.2 Data Flow Diagram……. 18

3. **Data Design**

3.1 DatabaseDescription…….....…19

4 **User Interface Design**

4.1Detailed Description of Components…… Page No

4.2 Screen images

5. TYPES OF TESTS (With Implementation)..... Page No

6. Appendices

7. References

1. Introduction

Vedic Human is an AI-based yoga training system designed to assist users in practicing yoga with correct posture. The system uses real-time pose detection through a webcam to analyse yoga postures and provide instant corrective feedback. It also allows users to track their yoga sessions and monitor progress over time through a web-based interface. This Software Requirements Specification (SRS) document describes the overall purpose, scope, and requirements of the Vedic Human system and serves as a reference for developers and stakeholders during the software development process.

1.1 Purpose

The purpose of this Software Design Specification (SDS) document is to describe the design and architecture of the **Vedic Human** system. This document translates the requirements defined in the Software Requirements Specification (SRS) into a structured design that guides developers, designers, and testers during implementation. The intended audience includes software developers, system architects, testers, and project evaluators.

1.2 Scope

The software product to be developed is **Vedic Human**, an AI-based web application for guided yoga practice. The system enables users to perform yoga sessions using real-time pose detection through a webcam and provides instant posture feedback and progress tracking.

The application is designed for beginners, students, and regular yoga practitioners to help improve posture accuracy and consistency. The system focuses on posture guidance and does not replace professional yoga trainers or provide medical advice. The scope of this software aligns with the objective of applying artificial intelligence in the health and fitness domain.

1.3 Definitions, Acronyms, and Abbreviations.

- **Vedic Human**:
  The AI-based yoga training web application developed to provide real-time posture guidance and progress tracking.
- **AI (Artificial Intelligence)**:
  The capability of a computer system to perform tasks that normally require human intelligence, such as pose detection and analysis.
- **Pose Detection**:
  A computer vision technique used to identify and track human body keypoints from images or live video.
- **MoveNet**:
  A pre-trained pose estimation model used for real-time human body keypoint detection.

- **TensorFlow**:
  An open-source machine learning framework used to implement and run the AI pose detection model.
- **SRS (Software Requirements Specification)**:
  A document that describes the functional and non-functional requirements of a software system.
- **Webcam**:
  A camera device used to capture live video input for yoga pose detection.
- **Frontend**:
  The user interface of the system developed using HTML, CSS, and JavaScript.
- **Backend**:
  The server-side logic of the system implemented using Python and Flask.
- **SQL (Structured Query Language)**:
  A language used to store, retrieve, and manage data in a relational database.

1.4 Overview

(1) This Software Requirements Specification (SRS) document describes the requirements of the **Vedic Human** system. The document is organized into several sections to provide a clear understanding of the system.

(2) Section 1 introduces the system and explains its purpose, scope, definitions, and overall structure of the document. Section 2 provides a general description of the system, including product functions, user characteristics, constraints, and interfaces. Section 3 details the specific functional and non-functional requirements of the system that guide the design and development process.

(3) This document serves as a reference for developers, designers, and testers throughout the software development lifecycle.

2. System Architectural Design

This section describes the overall architecture of the **Vedic Human** system and explains how different components of the system are organized and interact with each other. The architectural design provides a high-level view of the system structure and serves as a foundation for detailed design and implementation.

The Vedic Human system follows a **three-tier architecture**, which separates the application into presentation, application, and data layers. This architecture improves modularity, scalability, and maintainability of the system.

2.1 High-level Design Overview

The Vedic Human system is designed using a three-tier client–server architecture consisting of the following components:

1. PresentationLayer(Frontend)

This layer provides the user interface through which users interact with the system. It is developed using HTML, CSS, and JavaScript and runs in a web browser. It handles user inputs such as login, yoga session selection, and webcam access, and displays posture feedback and progress information.

2.Application Layer (Backend)

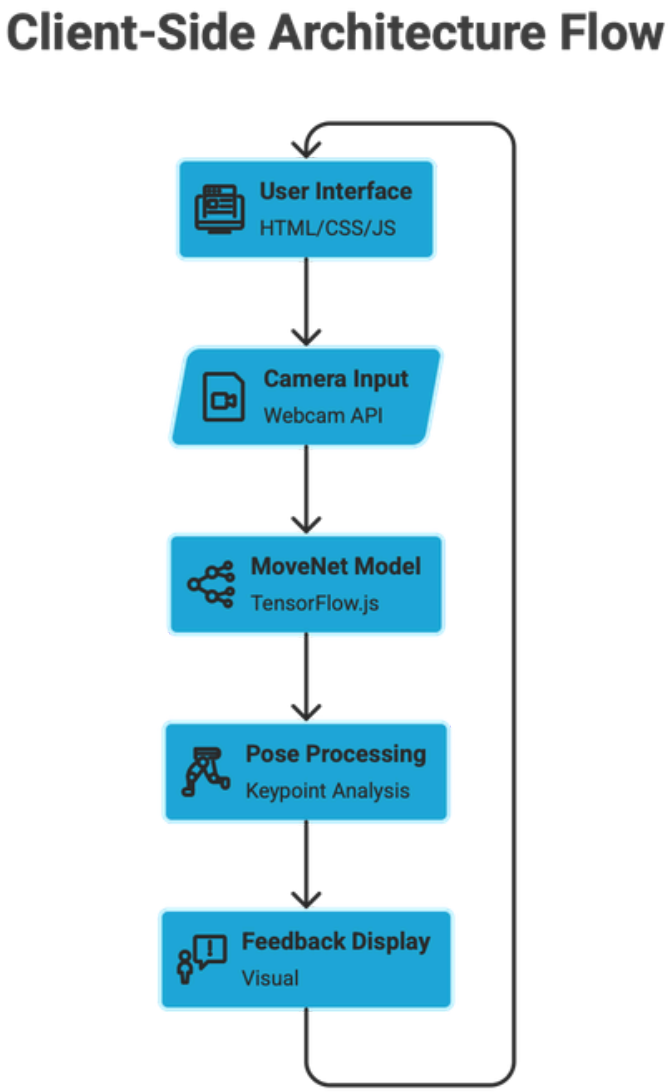
This layer contains the core logic of the system and is implemented using Python and Flask. It processes requests received from the frontend, manages user authentication, controls yoga sessions, communicates with the AI pose detection model, and handles data storage operations.

3.Data Layer (Database)

This layer stores persistent data such as user profiles, yoga session details, and performance records. A relational database such as SQLite or MySQL is used to ensure secure and efficient data management.

The frontend communicates with the backend using HTTP/HTTPS protocols, while the backend interacts with the database using SQL queries. The AI model is integrated into the backend to analyze video frames and generate posture evaluation results.

A pictorial representation of the system architecture is presented:



TThe client-side architecture consists of the user interface, webcam input, pose detection model, and feedback system. The user interacts through a web interface, and the webcam captures live video during a session.

The MoveNet model processes the video frames to extract body keypoints, which are analyzed to determine posture accuracy. Based on this analysis, real-time feedback is provided to the user through the interface.

2.2) Detailed Description of Components

The Vedic Human system is a web-based AI yoga training application consisting of multiple integrated components that work together to provide real-time posture analysis and feedback.

User Interface Component:

Provides an interactive interface for users to register, log in, start yoga sessions, and view progress. It captures webcam input and displays feedback and performance reports.

Backend Server Component:

Acts as the central controller of the system. It handles user authentication, session management, communication with the AI module, and database operations.

AI Pose Detection Component:

Processes live video frames using a machine learning model to detect body keypoints and evaluate yoga posture accuracy.

Feedback Generator Component:

Converts posture analysis results into real-time text and voice feedback to guide users during yoga sessions.

Progress Analysis Component:

Analyses stored session data to generate yoga progress reports and performance insights.

Database Component:

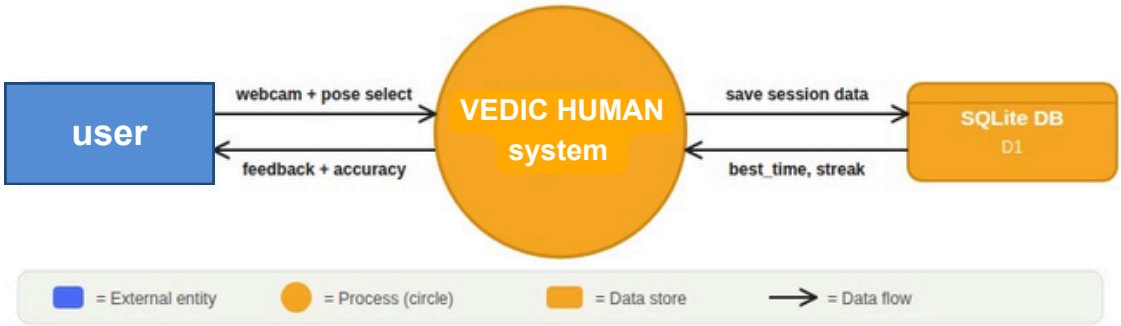
Stores user details, yoga session records, posture accuracy scores, and progress history.

Examples:-

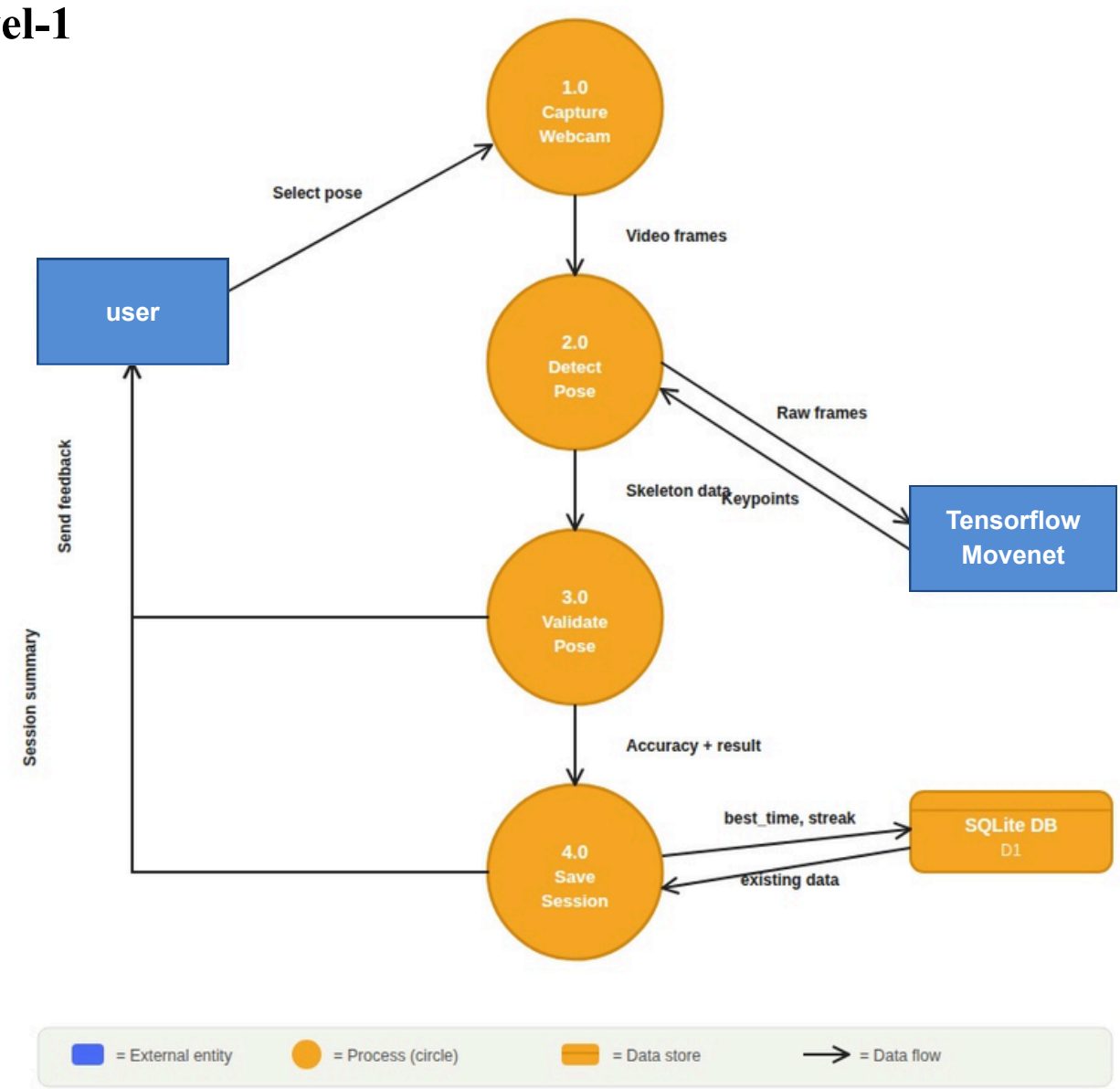
Data flow diagram (DFD) and Structure chart, Use Case diagram, Entity relationship diagram, Class diagram, Activity diagram and Sequence chart.

DFD diagram

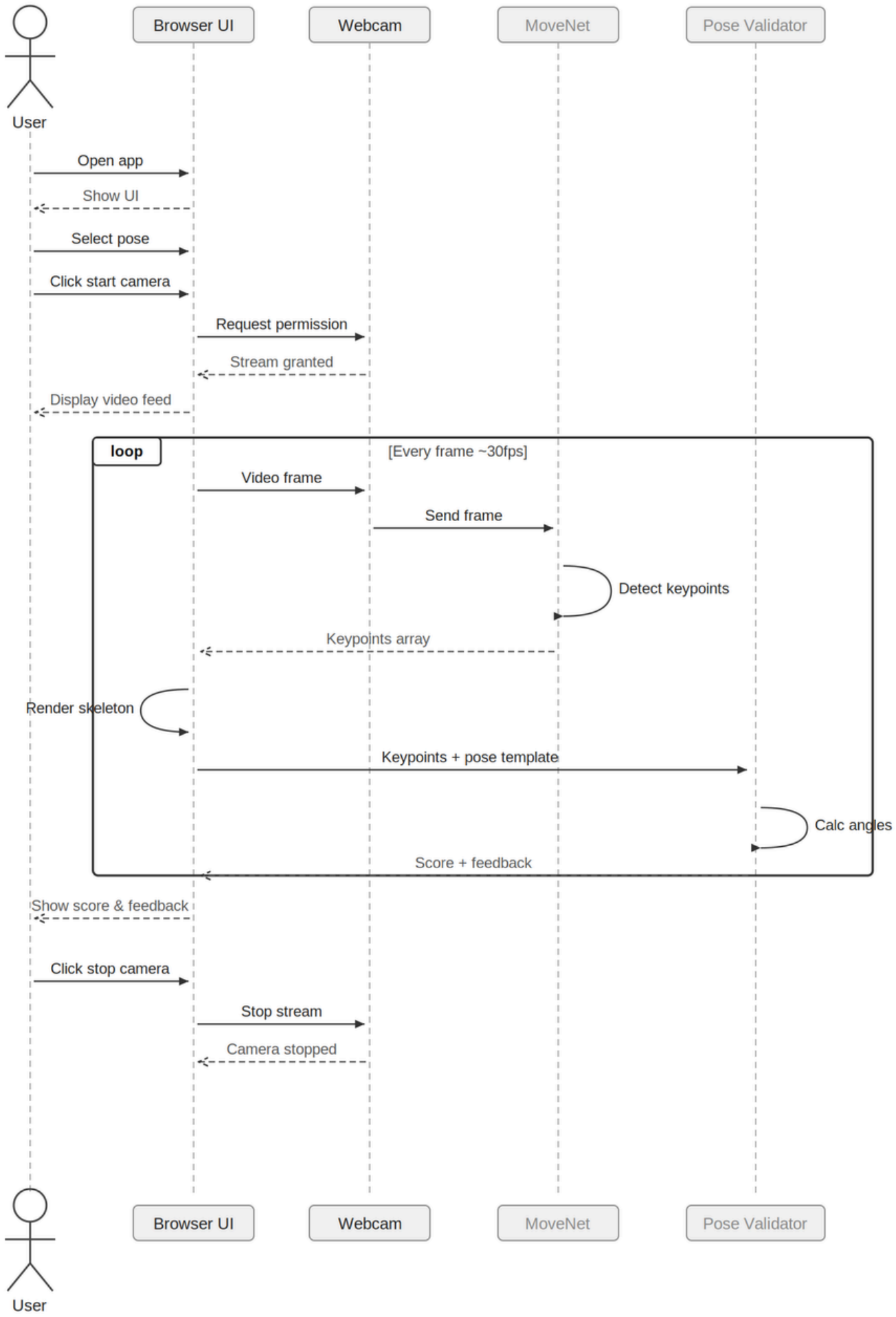
level-0



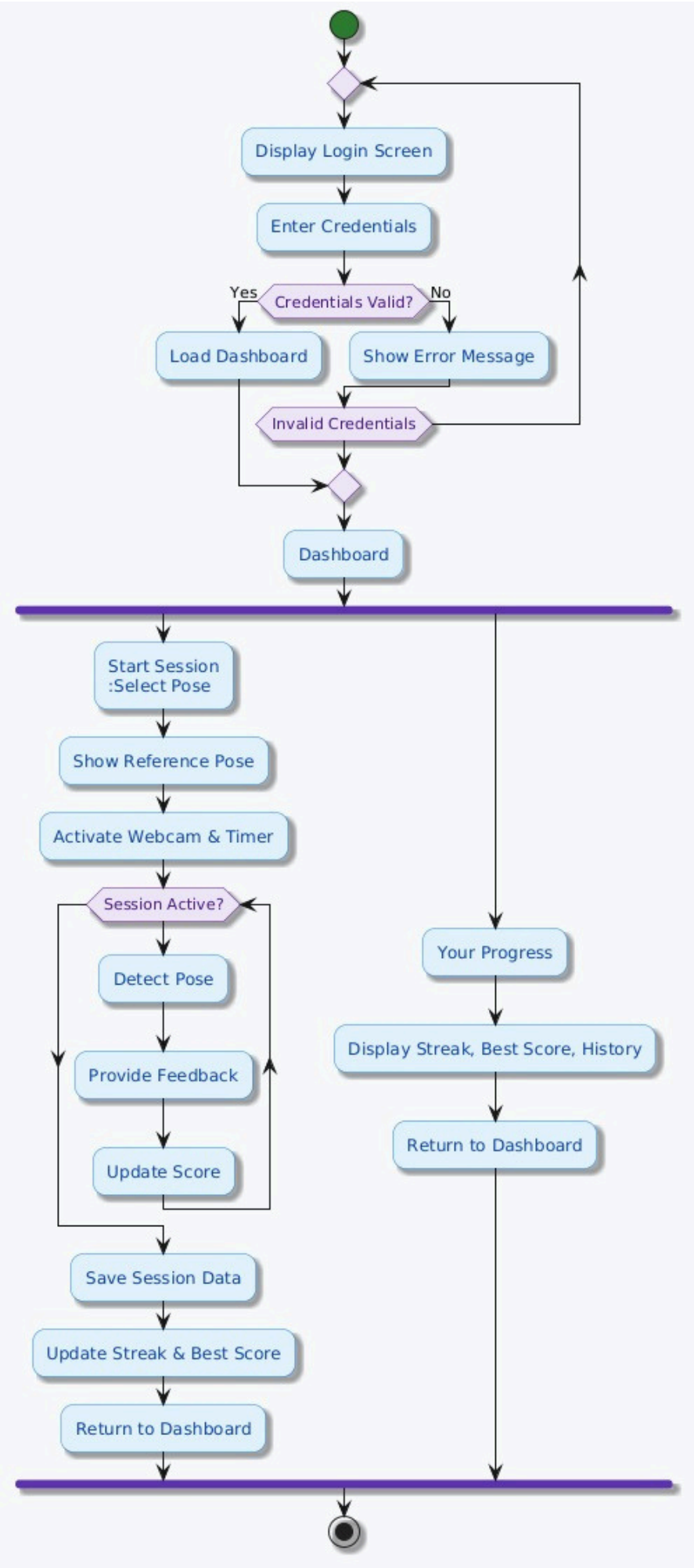
level-1



Sequence Diagram



ACTIVITY DIAGRAM



3. Structure and Relationships

The Vedic Human system follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Data Layer.

- The User Interface (Frontend) interacts with the user and sends requests to the backend.
- The Backend (Flask Server) acts as the central controller. It manages authentication, yoga sessions, and communication with the AI module and database.
- The AI Pose Detection Module (MoveNet) processes video frames and returns body keypoints for posture evaluation.
- The Database Layer (SQLite) stores user details, session data, and progress records.

All components are interconnected. The frontend communicates with the backend through HTTP requests. The backend interacts with the AI model for pose detection and with the database for storing and retrieving data.

This modular structure improves maintainability and scalability of the system.

3.1 Data Design

The system uses structured data storage to manage user information and yoga session records.

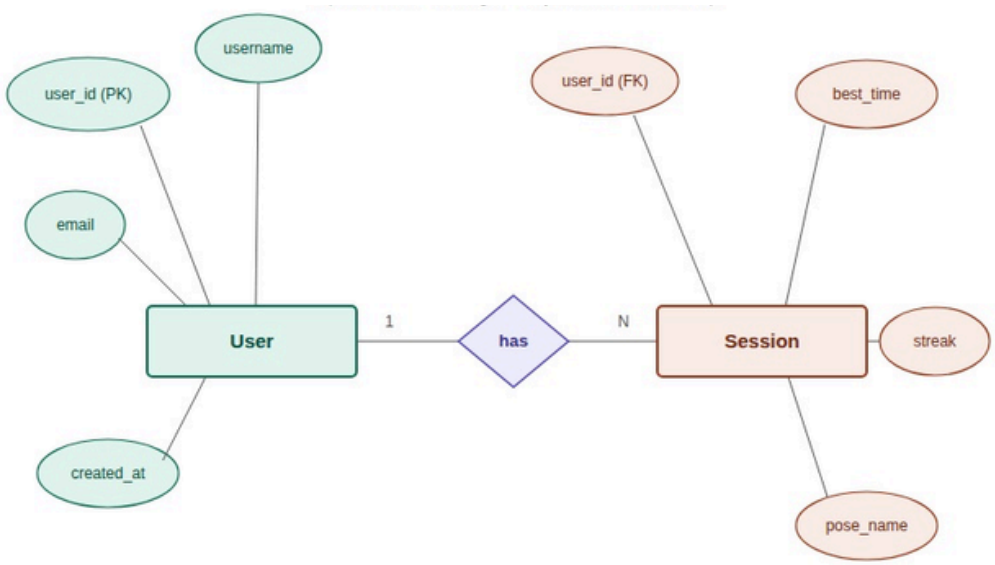
The major data structures used in the system are:

1. **User Data**
   - User ID (Primary Key)
   - Username
   - Email
   - Password

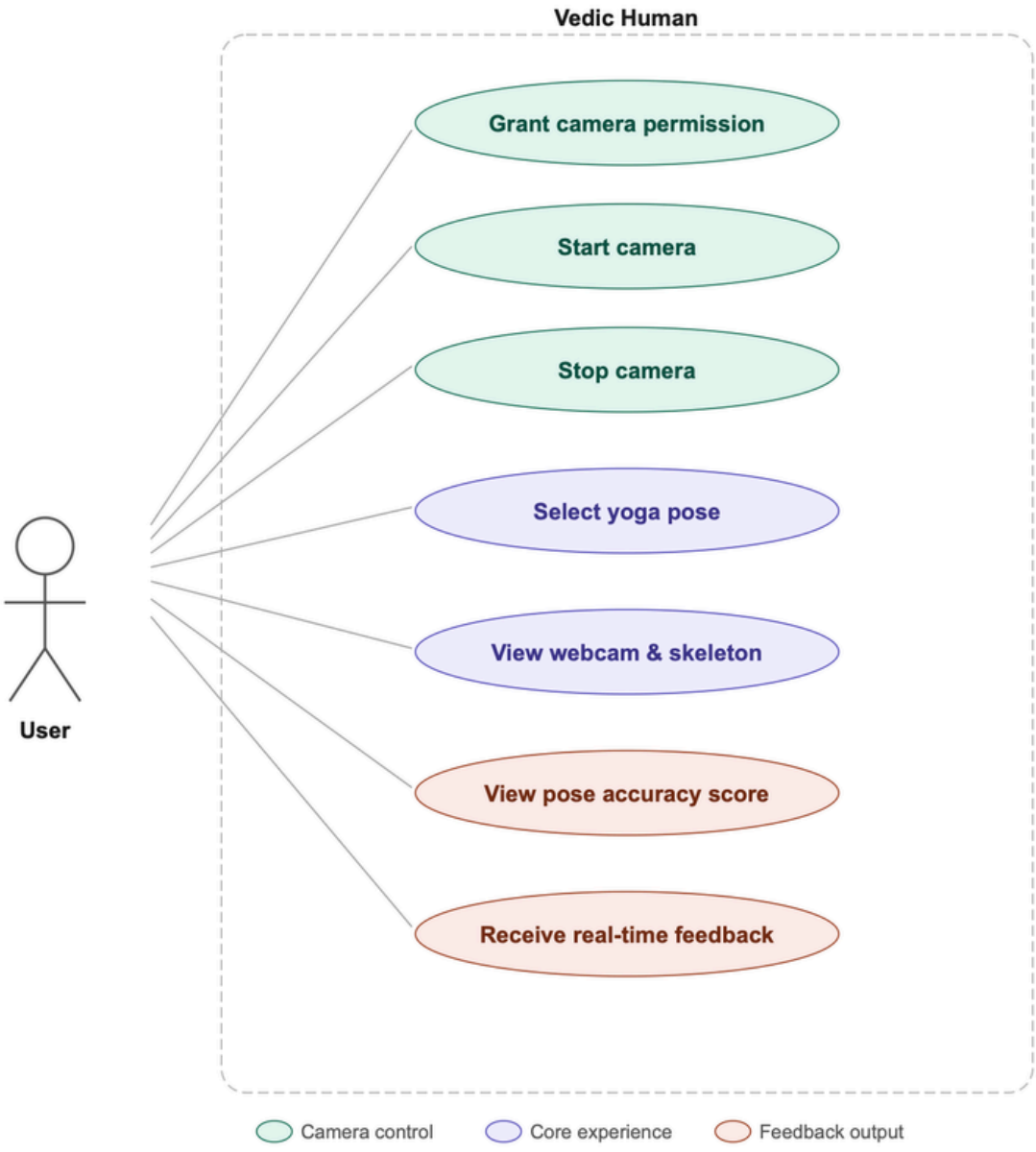
2. **Session Data**
   - User ID (Foreign Key)
   - Duration
   - Best Score
   - Session Date

The database is normalized to avoid data redundancy and ensure consistency.

ER Diagram - Website Database



Use Case Diagram



4. TYPES OF TESTS (With Implementation)

1. Unit Testing

Unit testing is performed to check individual modules of the system separately.

Modules Tested:

- Login Function
- Pose Detection Function
- Session Saving Function
- Progress Retrieval Function

Implementation:

- Backend functions were tested using sample inputs.
- Login was tested using valid and invalid credentials.
- Pose detection function was tested using sample video frames.
- Database insert and fetch queries were tested independently.

2. Integration Testing

Integration testing ensures that different modules work correctly when connected together.

Example:

- Frontend → Backend communication
- Backend → AI model communication
- Backend → Database interaction

Implementation:

- Tested whether login data entered in frontend is correctly verified by backend.
- Tested whether pose detection results are properly stored in database.
- Verified that progress page correctly displays stored session data.

.

3. System Testing

System testing checks the complete working of the application as a whole.

Implementation:

User login → Start session → Perform pose → Receive feedback → Save session → View progress.

Tested complete workflow in browser environment.

Checked system behavior under normal usage conditions.

4. Performance Testing

Performance testing ensures the system responds within acceptable time.

Implementation:

Verified that pose detection runs in real-time.

Measured response time of login and page loading.

Ensured UI loads within 2 seconds.

5. Security Testing

Security testing ensures user data protection.

Implementation:

Tested authentication with wrong password.

Ensured only logged-in users can access session page.

Passwords stored securely in database.

6. Stress Testing

Stress testing checks system performance under heavy usage.

Implementation:

Simulated multiple sessions.

Checked database behavior when many records are stored.

Observed system response during continuous pose detection

AcceptanceTestsandStaging

1. Acceptance Testing

Acceptance testing is performed to verify that the system meets all functional and non-functional requirements specified in the SRS document.

In this phase, the complete system is tested from the user's perspective.

Implementation:

Verified that user can successfully register and log in.

Checked that yoga session starts after login.

Confirmed real-time pose detection works using webcam.

Verified that posture feedback is displayed properly.

Checked that session data is stored correctly in database.

Ensured progress page displays accurate session history.

The system was tested according to the requirements listed in Section 3 (Functional Requirements). All major features worked as expected**.**

2. Staging

Staging environment is used before final deployment to test the system in an environment similar to production.

Implementation:

The system was tested on a local server using Flask.

Tested on different web browsers (Chrome, Edge).

Verified database operations under multiple sessions.

Checked overall system stability before final submission.

This ensured that the system performs correctly before final deployment.

5. Appendices

Appendix A: System Screens (Sample UI)
This appendix includes sample user interface screens such as:

- Login Page
- Dashboard
- Yoga Session Page
- Progress Page

These screens represent the layout and basic design of the Vedic Human system.
Appendix B: Database Tables
The main database tables used in the system are:

1. Users Table
   - user_id (Primary Key)
   - username
   - email
   - password
2. Sessions Table
   - session_id (Primary Key)
   - user_id (Foreign Key)
   - pose_name
   - duration
   - accuracy_score
   - date

Appendix C: Hardware and Software Requirements
Hardware Requirements:

- Laptop or PC with webcam
- Minimum 4GB RAM

Software Requirements:

- Web Browser (Chrome, Firefox, Edge)
- Python 3.x
- Flask
- TensorFlow
- SQLite

Appendix D: Assumptions

- Users have basic knowledge of using a web browser.
- Proper lighting and webcam positioning is required for accurate pose detection.
- Internet connection is required if the system is deployed online.

6. References

IEEE Standard for Software Requirements Specification (SRS), IEEE Std 830-1998.

Pressman, R. S. (2010). Software Engineering: A Practitioner's Approach. McGraw-Hill Education.

TensorFlow Official Documentation.
Available at: https://www.tensorflow.org

MoveNet Pose Detection Model Documentation.
Available at: https://www.tensorflow.org/hub/tutorials/movenet

Flask Official Documentation.
Available at: http://flask.palletsprojects.com/en/stable/

SQLite Documentation.
Available at: https://www.sqlite.org/docs.html

PlantUML Documentation (for UML diagrams).
Available at: https://plantuml.com

W3Schools – HTML, CSS and JavaScript Tutorials.
Available at: https://www.w3schools.com